CONCURRENCIA Y PARALELISMO

El problema del barbero dormilón

Planteamiento, solución con **semáforos** y pseudocódigo

Dijkstra, 1965 · Productor–consumidor con búfer finito · **ESPERAR / SEÑALIZAR**

INTEGRANTES DEL EQUIPO

Alfredo Ramírez Candia

Luis Antonio Islas Gómez

Navarro Soto Mario Alberto

Perez Flores Julio Cesar

Planteamiento del problema y solución con semáforos

La barbería

- **1 solo barbero** y una sala de espera con **N sillas**.
- Si no hay clientes, el barbero **duerme**.
- Llega un cliente:
  - Si el barbero duerme → lo **despierta**.
  - Si está ocupado → espera en una silla.
  - Si no hay sillas → **se va**.
- Al terminar: atiende al siguiente o **vuelve a dormir**.

El fallo sin sincronización

Cliente y barbero actúan **a la vez**: él se va a dormir justo cuando el cliente llega → **ambos duermen**.

Se perdió la señal de "hay trabajo": **lost wakeup**. También hay **carrera** sobre el contador de sillas.

Algoritmo con semáforos

Semáforo / recurso	Valor inicial	Qué coordina
`clientes`	0	Clientes esperando: **SEÑALIZA** el cliente al llegar, **ESPERA** el barbero (se duerme aquí).
`barbero_listo`	0	Barbero disponible: **SEÑALIZA** el barbero al despertar, **ESPERA** el cliente (no se sienta antes de ser atendido).
`sillas_libres`	N	Sala con **buffer finito**: **ESPERA** el cliente para ocupar silla, **SEÑALIZA** al salir.
`mutex`	1	Exclusión mutua sobre el **contador de sillas ocupadas** (región crítica).

`ESPERAR(s)` = decrementa y bloquea si llega a 0 · `SEÑALIZAR(s)` = incrementa y despierta a quien espera · Clásico patrón **productor–consumidor con búfer finito**.

Pseudocódigo de la solución

```
//INICIALIZACIÓN
SEMAFORO clientes      ← 0      // Barberia vacia
SEMAFORO barbero_listo ← 0      // barbero ocupado
SEMAFORO sillas_libres ← N      // N sillas vacías
MUTEX    ocupadas      ← 1      // proteger contador
ocupadas ← 0                    // sillas usadas

//PROCESO CLIENTE
INICIO cliente
  ESPERAR(sillas_libres)         //al entrar un cliente pregunta si hay silla
  ENTRAR EN ocupadas             // región crítica
     ocupadas ← ocupadas + 1
  SALIR DE ocupadas
  SEÑALIZAR(clientes)            // despierta al barbero
  ESPERAR(barbero_listo)         // espera su turno

  //recibe el corte de pelo

  ENTRAR EN ocupadas
     ocupadas ← ocupadas - 1
  SALIR DE ocupadas
  SEÑALIZAR(sillas_libres)       // deja libre la silla
FIN cliente
```

```
//PROCESO BARBERO
//(bucle infinito)
INICIO barbero  REPETIR PARA SIEMPRE

  ESPERAR(clientes)              // duerme si no hay
                                 // nadie esperando
  SEÑALIZAR(barbero_listo)       // avisa: estoy listo

  //corta el pelo al cliente

  // al terminar, si nadie espera,
  // ESPERAR(clientes) lo vuelve a
  // bloquear: el barbero duerme

FIN REPETIR
FIN barbero
```

¿Qué problemas reales soluciona en el ámbito computacional?

Es el patrón **productor–consumidor con búfer finito**: **colas de trabajos** de servidores web y de impresión, **pools de hilos** que atienden tareas de un usuario, **colas de mensajes** entre procesos, y el *backlog* de conexiones de un socket. El productor espera si el búfer está lleno, el consumidor duerme si está vacío: **mismo algoritmo, mismos semáforos**.